```
class Article < ActiveRecord::Base
  has_many :comments
  validates :title, presence: true,
                    length: { minimum: 5 }
end
```

These two declarations enable a good bit of automatic behavior. For example, if you have an instance variable `@article` containing an article, you can retrieve all the comments belonging to that article as an array using `@article.comments`.

For more information on Active Record associations, see the Active Record Associationsguide.

6.3 Adding a Route for Comments

As with the `welcome` controller, we will need to add a route so that Rails knows where we would like to navigate to see `comments`. Open up the `config/routes.rb` file again, and edit it as follows:

```
resources :articles do
  resources :comments
end
```

This creates `comments` as a *nested resource* within `articles`. This is another part of capturing the hierarchical relationship that exists between articles and comments.

For more information on routing, see the Rails Routing guide.

6.4 Generating a Controller

With the model in hand, you can turn your attention to creating a matching controller. Again, we'll use the same generator we used before:

```
$ bin/rails generate controller Comments
```

This creates five files and one empty directory:

File/Directory	Purpose
app/controllers/comments_controller.rb	The Comments controller
app/views/comments/	Views of the controller are stored here
test/controllers/comments_controller_test.rb	The test for the controller
app/helpers/comments_helper.rb	A view helper file
app/assets/javascripts/comment.coffee	CoffeeScript for the controller
app/assets/stylesheets/comment.scss	Cascading style sheet for the controller

Like with any blog, our readers will create their comments directly after reading the article, and once they have added their comment, will be sent back to the article show page to see their comment now listed. Due to this, our `CommentsController` is there to provide a method to create comments and delete spam comments when they arrive.

So first, we'll wire up the Article show template (`app/views/articles/show.html.erb`) to let us make a new comment:

```
<p>
  <strong>Title:</strong>
  <%= @article.title %>
</p>
```